

Viva Questions & Answers — Detailed Explanations

Deepfake Detection Using Cross-Modal Transformer Fusion

Jasmi Preethi Alasapuri | 2571395 | CN6000 Dissertation 2026

1. Project Overview & Motivation

Q1: Summarize your project in 2 minutes.

This project designed, implemented, and evaluated a multimodal audio-visual deepfake detection system using a Cross-Modal Transformer Fusion architecture. The system processes video frames through a ResNet3D-18 encoder pretrained on Kinetics-400 and audio through a ResNet18 encoder operating on mel-spectrograms. Both modalities are fused via a 2-layer Transformer Encoder with a learnable [CLS] token, producing three independent predictions — audio authenticity, video authenticity, and joint authenticity.

Implementation details: - **Codebase:** `main.py` (790 lines, pipeline orchestration), `cross_modal.py` → `TransformerFusion` class (249 lines), `audio.py`, `video.py`, `train_utils.py`, `data_utils.py`, `inference.py` (540 lines standalone). - **Dataset:** AV-Deepfake1M++ validation split — 77,326 metadata entries → 68,851 usable clips after filtering (211 zero-frame clips, 8,264 missing/corrupted on disk). Four manipulation types: real (~19K), audio_modified (~17K), visual_modified (~17K), both_modified (~17K). - **Speaker-disjoint partition:** `GroupShuffleSplit` on speaker IDs extracted from file paths (`data_utils.py` lines 128–129), verified zero overlap via assertion. 1,835 unique speakers in the validation split, split 80/20. - **Two-phase training:** Phase 1 (frozen encoders, ~2 epochs) at fusion LR $1e-4$, Phase 2 (unfrozen, encoder LR $1e-5$). Focal Loss ($\gamma=2.0$, $\alpha=0.25$), AdamW optimizer, ReduceLROnPlateau scheduler. - **Best model (Model 3):** Epoch 3 checkpoint, test AUC 0.937, accuracy 93.0%, precision 1.000, F1 0.837, zero false positives on 100-video test set. Wilson 95% CI: [86.3%, 96.6%] for overall accuracy. - **Deliverables:** 3 loadable checkpoints in `logs/logs_2/`, `logs/logs_3/`, `logs/logs_4/`; cloud-resumable pipeline; W&B audit trail; Flask web interface (Analyze/Compare/History tabs); standalone CLI inference.

Key finding: Score calibration determines deployment readiness more than raw AUC. Model 2 (AUC 0.994, 5 epochs) → 66% accuracy at $\tau=0.5$. Model 3 (AUC 0.985, 3 epochs) → 93% accuracy, zero false positives. The 0.009 AUC difference masks a 27-percentage-point accuracy gap. Reliability diagrams (`figures/calibration_curves.png`) confirm Model 3's calibration gap of 0.19 vs Model 2's 0.31.

Code locations: Architecture at `cross_modal.py:152-221`; training at `train_utils.py`; speaker split at `data_utils.py:128-135`; inference at `inference.py`; web at `web/app.py`.

Q2: What is the main contribution of your work?

Four contributions, each with specific implementation details:

1. Speaker-disjoint evaluation protocol (addressing identity leakage)

Rossler et al. (2019) demonstrated on FaceForensics++ that random train/validation splits inflate accuracy by 5–15 percentage points because models learn to recognise faces rather than detect manipulation artifacts. My implementation (`data_utils.py:128–135`):

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, val_idx = next(gss.split(df, groups=df['speaker']))
overlap = set(train_df['speaker'].unique()) & set(val_df['speaker'].unique())
assert len(overlap) == 0
```

Speaker IDs are extracted from HFS file paths (`source/id00015/clip.mp4` → `id00015`). This ensures all videos from one speaker are assigned exclusively to one partition. The test set creation (`create_test_data.py`) applies a second `GroupShuffleSplit` on the validation-subset speakers, producing an independent 100-video test set with 21 unique test speakers and zero overlap with training speakers. This two-level speaker-disjoint protocol means reported metrics (AUC 0.937, accuracy 93%) reflect manipulation detection on **entirely unseen identities** — a stricter evaluation than random-split protocols common in the literature (Cai et al., 2024; Yi et al., 2023).

2. Three-head interpretable architecture

Unlike the binary-output systems in Cai et al. (2024) and Yi et al. (2023), the three-head design (`cross_modal.py:192–221`) produces independent per-modality predictions. The dissociation pattern is empirically confirmed by per-type head scores (`draft.typ` line 1425–1428):

Manipulation Type	Audio Head Score	Video Head Score	Joint Score
audio_modified	0.375 (suppressed)	0.870 (elevated)	0.409
visual_modified	0.716 (elevated)	0.700 (suppressed)	0.304
both_modified	0.303 (suppressed)	0.305 (suppressed)	0.146
real	0.716 (elevated)	0.733 (elevated)	0.676

The audio and video head scores dissociate in the direction predicted by the manipulation type — `audio_modified` clips get low audio scores with high video scores, `visual_modified` clips get

the reverse. This validates that the architecture achieves genuine modality specialisation rather than collapsing to a single signal.

3. Empirical finding on score calibration vs AUC

Model 2 (epoch 5, val AUC 0.994, test accuracy 66%) vs Model 3 (epoch 3, val AUC 0.985, test accuracy 93%). The mechanism: extended fine-tuning causes score compression toward the decision boundary (Guo et al., 2017). Model 2’s scores are better ranked (higher AUC) but compressed near 0.5; Model 3’s scores are well-separated (better calibrated). Calibration analysis via reliability diagrams (figures/calibration_curves.png, generated by plot_calibration_curves.py) is rarely reported in prior multimodal detection work.

At optimal thresholds: Model 2 performs best at $\tau=0.795$ (87% accuracy, F1 0.764), Model 3 at $\tau=0.533$ (93% accuracy, F1 0.837). The finding that threshold recalibration recovers much of Model 2’s ranking quality demonstrates that AUC and calibration are complementary metrics.

4. Production-ready engineering

- **Resumable manifests** (data_utils.py:427-546): JSON-based progress tracking saving every 500 successes. Crash-resumable — terminated cloud instances resume from last saved manifest. Failed videos tracked in *_failed.json.
- **Checkpoint system** (checkpoint_utils.py): Saves model state, optimizer state, scheduler state, Python RNG, NumPy RNG, PyTorch RNG — full reproducibility on resume. WANDB_ID_PATH tracks run identity for W&B resume.
- **Standalone inference** (inference.py): 540 lines, zero training dependencies, redefines model classes inline. CPU/GPU compatible.
- **Web interface** (web/app.py, web/templates/index.html): Flask REST API (5 endpoints), SQLite history, model comparison, 500MB upload limit.

Q3: Why did you choose AV-Deepfake1M++ over other datasets?

Dataset characteristics: AV-Deepfake1M++ (Cai et al., 2025) contains ~2 million video clips across 2,000+ speakers with four manipulation types. Key differentiating features:

	DFDC (Dolhansky, 2020)	FaceForensics++ (Rossler, 2019)	FakeAVCeleb (Yi, 2023)	AV-Deepfake1M++ (Cai, 2025)
Modality	Video only	Video only	Audio-visual	Audio-visual

Feature	DFDC (Dolhansky, 2020)	FaceForensics++ (Rossler, 2019)	FakeAVCeleb (Yi, 2023)	AV-Deepfake1M++ (Cai, 2025)
Manipulation types	Binary (real/fake)	Binary (real/fake)	Binary (real/fake)	4 types (real, audio, visual, both)
Scale	100K+ clips	~4,000 videos	~20,000 clips	~2M clips
Speakers	3,426 paid actors	~1,000	500+	2,000+
Real-world perturbations	Limited	Minimal	Moderate	Compression, noise, scaling
Temporal annotations	No	No	No	fake_segments
Standardised benchmark	Kaggle competition	Research	Research	ACM MM 2025 challenge

Why the 4-type categorization matters: Binary labels (real/fake) tell you *that* a video is manipulated but not *what kind* of manipulation. The four-type scheme maps directly to the three-head architecture: `audio_modified` → low audio score + high video score; `visual_modified` → reverse; `both_modified` → both low; `real` → both high. This enables per-type evaluation and modality-level interpretability that binary-label datasets cannot support.

Practical constraint: The full training set is approximately 1.4 TB accessible via Hugging Face. As a student project without institutional GPU access, downloading and processing the full dataset within the project timeline (January–May 2026) was infeasible. I used only the validation split (68,851 usable clips, ~140 GB raw + ~35 GB extracted features = ~178 GB total). Storage fit within the 500 GB SSD of a Vast.ai instance. This constraint is explicitly acknowledged in dissertation Section 5.5 and limits the conclusions that can be drawn.

Data cleaning pipeline (`data_utils.py:94`, `eda.md`): 1. Initial metadata: 77,326 entries from `val_metadata.json`. 2. Stage 1: Remove entries with `audio_frames == 0` or `video_frames == 0` → 77,115 remaining (211 flagged). 3. Stage 2: Verify file presence on disk

→ 68,851 remaining (8,264 missing or corrupted, 89% yield). 4. Distribution after cleaning: real 20,220; visual_modified 19,099; both_modified 19,069; audio_modified 18,938.

Q4: How does your work differ from the AV-Deepfake1M++ baseline (DiMoDif)?

DiMoDif (Cai et al., 2025) is the official baseline detector for AV-Deepfake1M++, using cross-modal attention with temporal boundary detection. Detailed architectural comparison:

Aspect	DiMoDif (Baseline)	This Work	Impact of Difference
Fusion mechanism	Frame-aligned cross-modal attention → temporal boundary detection	Transformer Encoder (2 layers, 8 heads, 512-dim, GELU, pre-norm) with [CLS] token	My approach uses global self-attention over modality tokens rather than time-aligned cross-attention. This allows audio and video to attend to each other at every layer regardless of temporal position. Trade-off: loses temporal resolution but gains simpler architecture.
Output structure	Joint real/fake prediction + temporal boundary prediction	Three-head: audio (), video (), joint ()	The three-head output provides per-modality interpretability — you know <i>which modality</i> triggered the fake verdict. DiMoDif’s binary output cannot tell audio_modified from visual_modified without additional analysis.
Audio encoder	RawNet2-style (raw waveform → SincNet filters)	ResNet18 (mel-spectrogram → 2D CNN)	RawNet2 operates on raw waveforms; ResNet18 on mel-spectrograms needs FFT preprocessing but leverages ImageNet pretraining (11.7M pretrained weights). The mel-spectrogram approach proved more robust to the corrupted MP4 containers in AV-Deepfake1M++.

Aspect	DiMoDif (Baseline)	This Work	Impact of Difference
Loss function	Not specified in detail in Cai et al. (2025)	Focal Loss ($\alpha=2.0$, $\gamma=0.25$), joint head weighted $2\times$	Focal Loss explicitly handles class imbalance and easy-example domination. Without knowing DiMoDif’s loss, I cannot compare directly.
Training data	Full AV-Deepfake1M++ training split (1M+ clips)	Validation split only (68,851 clips)	DiMoDif has a data-scale advantage that prevents direct numerical comparison. My results should be interpreted as these-architecture-on-this-data-subset, not as competitive with DiMoDif.

The [CLS] token approach (`cross_modal.py:170`) follows BERT (Devlin et al., 2019). The token sequence [CLS, video_feat, audio_feat] with learnable positional embeddings allows the [CLS] token to aggregate cross-modal information through self-attention. After 2 Transformer layers, the [CLS] token’s representation feeds three independent Linear + Sigmoid heads. Total parameters: $\sim 49.7\text{M}$ (ResNet3D-18: 33.3M, ResNet18: 11.7M, Transformer: 4.7M, Heads: 1.5K).

What I cannot claim: Direct performance comparison with DiMoDif. Different training data regimes (validation split vs full training set), different evaluation protocols, and different computational budgets make numerical comparison invalid. The architectural differences are qualitative contributions, not quantitative improvements over the baseline.

2. Literature Review

Q5: What are the key gaps you identified, and how does your work address them?

Three gaps from dissertation Section 2.7, with detailed evidence and solutions:

Gap 1 — Identity Leakage

Evidence: Rossler et al. (2019) trained XceptionNet on FaceForensics++ and demonstrated that models exploiting face identity rather than manipulation artifacts achieve artificially high accuracy. When the same person appears in both training and test sets (as happens with random splits), the model learns to recognise the person rather than detect the manipulation. Dolhansky et al. (2020) found that DFDC models with access to same-identity data across splits inflated AUC by 0.05–0.10 compared to identity-disjoint evaluation.

My solution: `GroupShuffleSplit` on speaker IDs extracted from file paths (`data_utils.py:120-135`). Speaker IDs follow the pattern `source/id00015/clip.mp4` → extracted as `id00015`. Two-level enforcement: 1. Main training/validation split: 1,468 train speakers, 367 validation speakers, verified zero overlap. 2. Test set (`create_test_data.py`): Independent `GroupShuffleSplit` on validation-subset speakers, producing 105 train speakers and 27 test speakers (yielding 21 unique test speakers in the 100-video set).

Result: All reported AUC and accuracy metrics reflect generalisation to entirely unseen speaker identities. This is a stricter evaluation than random-split protocols used in most prior multimodal detection work.

Gap 2 — Vision-Centric Bias

Evidence: Yi et al. (2023) surveyed multimodal audio-visual detection and found that most systems use simple feature concatenation for fusion. Audio features are treated as supplementary channels appended to visual features, rather than as equal modalities. This prevents the model from learning fine-grained audio-visual correspondences — for example, temporal alignment between phonemes in speech and visemes in lip motion.

My solution: Transformer Encoder (`cross_modal.py:152-221`) where the 3-token sequence `[CLS, video_feat, audio_feat]` undergoes multi-head self-attention at every layer. The $Q \cdot K^T$ attention matrix (3×3 per head) allows each token to attend to all others — video attends to audio, audio attends to video, `[CLS]` attends to both. This is architecturally different from concatenation, which only allows post-hoc fusion. The 2-layer depth means attention is applied twice, allowing second-order interactions (e.g., video attends to audio-attended-to-CLS).

Evidence of effectiveness: The modality dissociation pattern (Section 4.4, `draft.typ` lines 1425–1428) shows that `audio_modified` clips produce suppressed audio head scores and elevated video head scores, while `visual_modified` clips produce the reverse. This directional specialisation would not emerge from simple concatenation fusion.

Gap 3 — Limited Generalisation

Evidence: Dolhansky et al. (2020) evaluated DFDC-winning models on held-out datasets and found AUC degradation of 0.10–0.30 when applied to videos from unseen generators. This is a well-documented problem across the detection literature — detectors overfit to dataset-specific artifacts rather than learning generator-invariant manipulation signatures.

My partial solution: The speaker-disjoint split addresses identity generalisation (unseen speakers) but does not address generator generalisation (unseen manipulation techniques). The model was trained and evaluated entirely on AV-Deepfake1M++, so it may overfit to artifacts specific to the TTS/VC systems and NeRF-based video generators used in that dataset. Cross-dataset evaluation on FakeAVCeleb or DFDC — testing generalisation to different generators, different recording conditions, and different compression codecs — is the top-priority future work and the most practically relevant measure of detector reliability.

Q6: Why is speaker-disjoint splitting important? Quantify the impact.

The problem: When the same speaker appears in both training and test splits (as happens with random splitting), a model can learn to recognise faces and voices rather than detect manipulation artifacts. This conflates two different capabilities (identity recognition and manipulation detection) into a single metric.

Published evidence: - Rossler et al. (2019): XceptionNet on FaceForensics++ with random splits achieved >0.95 accuracy. Under identity-disjoint evaluation, accuracy dropped by 5–15 percentage points depending on manipulation method. - Dolhansky et al. (2020): DFDC models with same-identity data across splits showed 0.05–0.10 higher AUC than identity-disjoint evaluation. - Cai et al. (2024): AV-Deepfake1M baselines reported AUC values that likely include identity leakage effects (random-split evaluation).

My implementation (data_utils.py:120–135):

```
df['speaker'] = df['file'].apply(lambda f: f.split('/')[1] if '/' in f else 'unknown')
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)
train_idx, val_idx = next(gss.split(df, groups=df['speaker']))
train_df, val_df = df.iloc[train_idx], df.iloc[val_idx]
overlap = set(train_df['speaker'].unique()) & set(val_df['speaker'].unique())
assert len(overlap) == 0, f"Speaker overlap detected: {overlap}"
```

Scale: 1,835 unique speakers. Split: 1,468 train (80%), 367 validation (20%). All videos from a given speaker go to exactly one partition — if speaker X has 100 videos, all 100 go to train OR all 100 go to validation.

What I cannot quantify: Within this project, I did not run a controlled comparison of random vs speaker-disjoint splits (for example, training the same architecture with both splitting strategies and comparing test performance). This is a recognised limitation. The expected impact based on published literature is 5–15% accuracy inflation from random splits, but I cannot produce a project-specific number without the controlled experiment — which is future work.

Two-level enforcement: The main pipeline splits at 80/20. The test set (create_test_data.py) applies a second GroupShuffleSplit on the validation-subset speakers (367 → 105 train + 27 test), yielding 21 unique test speakers in the 100-video evaluation set — zero overlap with any training speaker at either level.

Q7: How have deepfake detection approaches evolved?

Three phases synthesised from the dissertation literature review (Chapter 2):

Phase 1 — Visual Fidelity (2017–2019): - **Generation:** Autoencoders with shared encoder + identity-specific decoders. Simple GAN-based face swaps (Li and Lyu, 2018). - **Detection:** Handcrafted forensic features — face warping boundaries, inconsistent eye blinking, color mismatch. CNN-based detectors (XceptionNet, MesoNet) trained on small datasets (FaceForensics++: ~4,000 videos, Rossler et al., 2019). - **Key limitation:** Detectors overfit to low-level artifacts (resolution, compression) that were dataset-specific, not generator-specific. - **Benchmark AUC:** 0.95–0.99 on FaceForensics++, dropping to 0.60–0.80 on cross-dataset evaluation.

Phase 2 — In-the-Wild (2019–2021): - **Generation:** Multi-subject scenes, uncontrolled lighting and backgrounds. Celeb-DF (Li et al., 2020) with 5,600 celebrity face-swaps. DFDC (Dolhansky et al., 2020) with 100K+ clips from 3,426 paid actors, multiple generation methods. - **Detection:** Temporal modeling via 3D CNNs (ResNet3D, I3D) and RNN + CNN hybrids (Güera and Delp, 2018). Attention mechanisms. Ensemble approaches winning DFDC (log-loss <0.20). - **Key finding:** Cross-dataset performance degradation of 0.10–0.30 AUC documented in Dolhansky et al. (2020). Detectors learned dataset-specific patterns, not manipulation signatures. - **Emerging datasets:** WildDeepfake (Zi et al., 2021), DF-Platter for multi-face occlusion handling (Narayan et al., 2023).

Phase 3 — Multimodal (2023–2025): - **Generation:** Neural TTS systems (NaturalSpeech 2, Shen et al., 2023) producing indistinguishable synthetic speech. NeRF-based talking heads (AD-NeRF, Guo et al., 2021) with controllable pose and lighting. These systems enable simultaneous audio AND video manipulation — the deepfake can fake both what you hear and what you see. - **Datasets:** FakeAVCeleb (~20K clips, 500+ speakers, multiple TTS/VC methods). AV-Deepfake1M (Cai et al., 2024, 1M clips). AV-Deepfake1M++ (Cai et al., 2025, 2M clips, 2,000+ speakers, 4 manipulation types, real-world perturbations). - **Detection requirement:** Cross-modal architectures that jointly analyse audio-visual coherence. The gap between audio and visual streams — unnatural lip-sync, spectral inconsistency, motion discontinuity — is now the primary detection signal. - **This project’s position:** Operates at Phase 3, using the largest available multimodal benchmark with a 4-type categorization that maps to per-modality interpretable output.

3. Architecture & Design

Q8: Explain your architecture in full detail.

The Cross-Modal Transformer Fusion network (`cross_modal.py` → `TransformerFusion`, lines 152–221) has 49.7M total parameters across three stages:

Stage 1 — Parallel Modality Encoding (45.0M params combined):

Video Encoder (`video.py`, ~33.3M params): - **Input:** 50 frames at $224 \times 224 \times 3$, sampled from 2 seconds at 25 fps. Tensor shape: (B, 50, 3, 224, 224) for 3D convolution — the time dimension is treated as the channel dimension for ResNet3D's ($3 \times 3 \times 3$) kernels. - **Architecture:** ResNet3D-18 (Tran et al., 2018), a 3D variant of ResNet18 where all 2D conv kernels (3×3) are replaced with 3D kernels ($3 \times 3 \times 3$). The network has 18 layers in 4 stages with residual connections. - **Pretraining:** Kinetics-400 (Kay et al., 2017) — 400 human action categories, 306,245 videos. This pretraining is relevant because the dataset includes talking, singing, gesturing, and other human motion relevant to lip-sync analysis. - **Output:** Global average pooling over spatial and temporal dimensions → Linear(512 → 256) → Dropout(0.4) → 256-d feature vector. - **Why 3D:** 2D CNNs process individual frames independently, detecting spatial artifacts (blur, facial boundary). 3D CNNs process frame sequences jointly, detecting temporal artifacts — unnatural acceleration in lip motion, micro-jitter between consecutive synthesized frames, motion patterns that don't match natural speech-driven articulation.

Audio Encoder (`audio.py`, ~11.7M params): - **Input:** Raw 16kHz mono audio, 2 seconds = 32,000 samples. Transformed to mel-spectrogram: FFT with 1024-point window, 512-sample hop, 128 mel frequency bins → (B, 1, 128, 63). 63 time steps = $(32000 - 1024) / 512 + 1$. - **Architecture:** ResNet18 (He et al., 2015), pretrained on ImageNet. First convolutional layer modified: `conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3)` — changes from 3 input channels (RGB) to 1 channel (grayscale spectrogram). - **Output:** Global average pooling → Linear(512 → 256) → Dropout(0.4) → 256-d feature vector. - **Why mel-spectrogram:** Compresses 32,000 samples into $128 \times 63 = 8,064$ values. The mel scale approximates human auditory perception, emphasising low frequencies where most speech energy resides. This representation is 2D (frequency × time), allowing a standard 2D CNN to process it.

Stage 2 — Transformer Cross-Modal Fusion (4.7M params):

This stage is implemented in `TransformerFusion.__init__()` (lines 160–194):

```
self.audio_proj = nn.Linear(feature_dim, hidden_dim) # 256 → 512
self.video_proj = nn.Linear(feature_dim, hidden_dim) # 256 → 512
self.cls_token = nn.Parameter(torch.randn(1, 1, hidden_dim))
self.pos_embedding = nn.Parameter(torch.randn(1, 3, hidden_dim))
encoder_layer = nn.TransformerEncoderLayer(
    d_model=hidden_dim, nhead=8,
```

```

        dim_feedforward=hidden_dim * 4, # 2048
        dropout=0.4, activation='gelu',
        batch_first=True, norm_first=True # Pre-LN
    )
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=2,
    norm=nn.LayerNorm(hidden_dim))

```

Forward pass (lines 196–221): 1. Project: $v = \text{self.video_proj}(\text{video_feat}).\text{unsqueeze}(1) \rightarrow (B, 1, 512)$ 2. Project: $a = \text{self.audio_proj}(\text{audio_feat}).\text{unsqueeze}(1) \rightarrow (B, 1, 512)$ 3. CLS token: $\text{cls} = \text{self.cls_token.expand}(B, -1, -1) \rightarrow (B, 1, 512)$ 4. Sequence: $\text{tokens} = \text{torch.cat}([\text{cls}, v, a], \text{dim}=1) \rightarrow (B, 3, 512)$ 5. Positional encoding: $\text{tokens} = \text{tokens} + \text{self.pos_embedding}$ 6. Transformer: $\text{fused} = \text{self.transformer}(\text{tokens}) \rightarrow (B, 3, 512)$ 7. Extract CLS: $\text{cls_out} = \text{fused}[:, 0, :] \rightarrow (B, 512)$

Why pre-norm (`norm_first=True`): Standard in modern Transformer architectures (LLaMA, GPT-3). LayerNorm is applied before attention/FFN sublayers rather than after. This improves training stability and gradient flow, which matters for the small 3-token sequence size.

Why 8 heads $\times$ 64-dim each: The multi-head attention splits the 512-d representation into 8 parallel 64-d subspaces. Each head can attend to different aspects of cross-modal relationship — one head might focus on temporal correspondence, another on spectral-visual alignment, another on identity consistency.

Why [CLS] token: Following BERT (Devlin et al., 2019), a special learnable token prepended to the input sequence. Through self-attention, this token attends to all other tokens (video and audio features) and aggregates their information. Its final hidden state serves as the fused cross-modal representation. This is architecturally elegant: the same mechanism that relates video to audio also produces the classification representation, without separate pooling or aggregation steps.

Stage 3 — Three-Head Classification (~1.5K params):

```

self.audio_classifier = nn.Linear(hidden_dim, 1) # 512  $\rightarrow$  1  $\rightarrow$ 
self.video_classifier = nn.Linear(hidden_dim, 1) # 512  $\rightarrow$  1  $\rightarrow$ 
self.joint_classifier = nn.Linear(hidden_dim, 1) # 512  $\rightarrow$  1  $\rightarrow$ 

```

All three heads receive the same [CLS] output and produce sigmoid-activated scores in (0, 1). The joint head labels use AND logic: $\text{joint_label} = \text{audio_label} \ \& \ \text{video_label}$ — a video is “real” only if both modalities are authentic.

Loss function (`train_utils.py` $\rightarrow$ `FocalLoss`):

```
L_total = 0.4 * L_audio + 0.4 * L_video + 0.8 * L_joint
```

The joint head receives 2× weight (=0.8 vs 0.4) because it is the primary deployment output.

Q9: Why did you replace Wav2Vec 2.0 with ResNet18? Was it the right decision?

Original plan: Wav2Vec 2.0 (Baevski et al., 2020) — a self-supervised speech representation model pretrained on 960 hours of LibriSpeech. Produces 768-d contextual embeddings per 20ms frame. Expected advantage: captures phoneme-level speech content, speaker identity, and prosodic information that would be sensitive to voice cloning (TTS/VC) artifacts. The literature (Yi et al., 2023) had shown promising results with self-supervised speech representations for audio deepfake detection.

The failure mode: The AV-Deepfake1M++ dataset contains non-standard MP4 containers with variable codecs. When librosa attempted to load audio: - **Silent failure:** `librosa.load()` returned `audio = np.array([])` with zero length and no error. The feature extraction pipeline would proceed with empty audio, producing NaN mel-spectrograms that caused the Wav2Vec encoder to output NaN features. - **Detection difficulty:** These failures were not detected at load time — they propagated silently through the pipeline and corrupted the extracted features. Only post-hoc analysis of feature statistics revealed the issue. - **Scale of impact:** Out of 77,326 metadata entries, 8,264 files were missing or corrupted on disk. The zero-audio-frame issue affected 211 entries. Combined, approximately 11% of the dataset was unusable.

Why torchaudio/FFmpeg was the fix: - `torchaudio.load(video_path, backend="ffmpeg")` — FFmpeg backend handles the MP4 container correctly, extracting the audio stream regardless of codec quirks. - Mel-spectrogram generation uses `torchaudio.transforms.MelSpectrogram` with the same FFT parameters as training (1024-point window, 512-sample hop, 128 mel bins). - ResNet18 processes the 2D spectrogram through standard 2D convolutions — pretrained ImageNet weights provide strong initial features even for single-channel (grayscale) input.

Evidence for “right decision”: 1. **Pipeline robustness:** 68,851 clips processed successfully vs the alternative of losing a significant fraction to Wav2Vec loading failures. 2. **Meaningful audio head discrimination:** The audio head produces directionally correct scores — `audio_modified` clips get mean audio scores of 0.375 vs 0.716 for real clips (Model 3). The dissociation pattern exists and is detectable. 3. **Training efficiency:** ResNet18 (11.7M params) is much smaller than Wav2Vec 2.0 BASE (95M params) or LARGE (317M params). This reduced GPU memory requirements and training time. 4. **Precedent:** Mel-spectrogram + ResNet approaches are standard in audio classification literature and were used in ASVspoof challenge submissions.

Caveats: - **Cannot claim superiority:** Without a controlled comparison (same training data, same architecture except audio encoder, same random seed), I cannot say ResNet18 is *better* — only that it was *practically viable*. - **Audio head weaker than video head:** The audio head’s discriminative margin is smaller than the video head’s (this is visible in the training history AUC curves and per-type score tables). Whether this is due to encoder choice or inherent difficulty of audio deepfake detection is unknown. - **Missed opportunity:** Wav2Vec 2.0’s contextual representations might have captured prosodic inconsistencies (unnatural intonation, rhythm) that mel-spectrograms miss. This hypothesis is untestable without the controlled experiment.

Q10: Why ResNet3D-18 instead of 2D CNN for video?

The fundamental limitation of 2D CNNs for video deepfake detection: 2D CNNs process each frame as an independent image. Frame 1 $\rightarrow$ features, Frame 2 $\rightarrow$ features, ..., Frame 50 $\rightarrow$ features. These features are then aggregated — typically by averaging or an LSTM. This architecture can detect: - Spatial artifacts: blur, unnatural skin texture, facial boundary inconsistencies. - Per-frame quality differences between real and synthetic regions.

But it cannot detect: *how pixels move between frames*. Lip-synced deepfakes synthesize the lip region in each frame to match a target audio track. Each individual frame may look spatially perfect — the lip shape matches the expected phoneme, the texture is realistic, the boundary is seamless. The manipulation is in the *trajectory* of lip movement — the sequence of shapes across consecutive frames. Some specific artifacts: - **Unnatural acceleration:** Real lip motion accelerates and decelerates smoothly following speech dynamics. Synthesized lip motion may show sudden jumps in velocity. - **Micro-jitter:** Frame-to-frame positional noise in the synthesized lip region — pixels jitter by 1–2 pixels between consecutive frames. - **Temporal discontinuity:** The transition between synthesized and real frames (when manipulation starts/ends) may show abrupt changes in motion pattern.

Why 3D convolutions detect these: A 2D conv kernel of size (3×3) operates on a single frame at spatial position (h, w) . A 3D conv kernel of size $(3 \times 3 \times 3)$ operates on three consecutive frames at spatial position (h, w) across time steps $(t, t+1, t+2)$. The kernel learns spatiotemporal patterns — it “sees” how pixel values change across time.

ResNet3D-18 (Tran et al., 2018) replaces all 2D (3×3) kernels in ResNet18 with 3D $(3 \times 3 \times 3)$ kernels. The network has 5 temporal downsampling operations (stride-2 in first conv layer and first layer of stages 2–5), meaning the deepest layers operate on highly temporally compressed features that capture motion patterns across the full 50-frame window.

Kinetics-400 pretraining (Kay et al., 2017): 306,245 videos across 400 human action categories including “talking,” “singing,” “playing musical instruments” — categories involving mouth and facial movement. This pretraining provides features relevant to lip motion analysis.

Why not MobileNetV3 (the initial plan): MobileNetV3 (Howard et al., 2019) is a 2D CNN designed for mobile efficiency using depthwise separable convolutions and squeeze-and-excitation blocks. It achieves strong ImageNet accuracy (75.2% top-1) with only 5.4M parameters. Advantages: fast inference, low memory, good spatial feature extraction. The limitation: no temporal modeling whatsoever. The temporal aggregation would need to be handled externally (LSTM, temporal pooling), and the 2D features would need to capture motion implicitly — which they are not designed to do.

Implementation (video.py):

```
model = torch.hub.load('pytorch/vision', 'resnet3d_18',
    weights='ResNet3D18_Weights.KINETICS400_V1')
model.fc = nn.Sequential(
    nn.Dropout(0.4),
    nn.Linear(512, feature_dim) # 512 → 256
)
```

`nn.Dropout(0.4)` is inserted before the projection layer for regularization.

Q11: Explain Focal Loss — mathematical and practical justification.

Formula and mechanism:

$$FL(p_t) = - \alpha_t \cdot (1-p_t)^{\gamma} \cdot \log(p_t)$$

where: $\alpha_t = p$ if $y=1$, else $1-p$ (model's confidence in the correct class) - $\gamma = 2.0$ (focusing parameter — how much to down-weight easy examples) - $\alpha_t = 0.25$ for class 1 (real), 0.75 for class 0 (fake) — class balance weight

How the modulating factor $(1-p_t)^{\gamma}$ works:

Scenario	p_t	$(1-p_t)^2$	Loss contribution
Confident & correct	0.99	$(0.01)^2 = 0.0001$	Near zero — example is “solved”
Moderately confident	0.80	$(0.20)^2 = 0.04$	Down-weighted 25×
Uncertain boundary case	0.50	$(0.50)^2 = 0.25$	4× less than BCE equivalent
Confident & wrong	0.01	$(0.99)^2 = 0.98$	Nearly full weight — hard example

The quadratic down-weighting means that as soon as the model becomes even moderately confident ($p_t > 0.7$), the loss contribution drops sharply. This concentrates gradient updates on the ambiguous cases — exactly the `audio_modified` clips where the joint head struggles to reconcile conflicting modality signals.

Why this matters for deepfake detection specifically:

1. **Many easy real videos:** A real video where both audio and video are clearly authentic (high-quality recording, clear speaker, stable lighting) will produce p_t close to 1.0 early in training. Without Focal Loss, these continue to dominate the gradient, preventing the model from focusing on hard cases.
2. **Hard `audio_modified` cases:** An `audio_modified` video where the video is genuine but audio is fake produces conflicting signals:
 - Video head: “this looks real” $\rightarrow$ high confidence ($p_{\text{video}} \approx 0.85$)
 - Audio head: “this sounds fake” $\rightarrow$ low confidence ($p_{\text{audio}} \approx 0.25$)
 - Joint head: must reconcile these $\rightarrow$ uncertain ($p_{\text{joint}} \approx 0.5$) The joint head prediction near 0.5 means $(1-p_t)^2$ is large, preserving the gradient signal on these ambiguous cases.
3. **Both `modified` cases start easy, become hard:** Early in training, both `modified` videos are easy (both modalities provide fake evidence). As training progresses and the model learns to detect per-modality artifacts, these become confidently classified. Focal Loss automatically shifts attention to the remaining hard cases.

Implementation (`train_utils.py` $\rightarrow$ `FocalLoss` class):

```
class FocalLoss(nn.Module):
    def __init__(self, gamma=2.0, alpha=0.25):
        super().__init__()
        self.gamma = gamma
        self.alpha = alpha

    def forward(self, pred, target):
        pred = torch.clamp(pred, 1e-6, 1.0 - 1e-6) # Numerical stability
        bce = -(target * torch.log(pred) + (1 - target) * torch.log(1 - pred))
        p_t = pred * target + (1 - pred) * (1 - target) # p if y=1 else 1-p
        focal_weight = (1 - p_t) ** self.gamma
        alpha_t = self.alpha * target + (1 - self.alpha) * (1 - target)
        return (alpha_t * focal_weight * bce).mean()
```

The `torch.clamp(pred, 1e-6, 1.0 - 1e-6)` prevents `log(0)` which would produce NaN gradients.

Combined loss (`train_utils.py`):

```
loss = (0.4 * focal_loss(audio_pred, audio_label) +  
        0.4 * focal_loss(video_pred, video_label) +  
        0.8 * focal_loss(joint_pred, joint_label))
```

The joint head receives $2\times$ weight because it is the primary deployment output. Audio and video heads receive auxiliary loss (40% each) to encourage modality-specific feature learning.

Without a controlled ablation: I cannot attribute the zero-false-positive performance of Model 3 to Focal Loss specifically. A controlled experiment training the same architecture with BCE under identical conditions would be needed. The observed rapid convergence (AUC ~ 0.99 by epoch 2) and clean score separation are consistent with Focal Loss behavior, but correlation is not causation.

Q12: Why three heads instead of binary output?

Purpose 1 — Interpretability for forensic analysis:

A binary detector says “this video is 87% likely fake.” A three-head detector says: - Audio: 23% authentic (likely fake audio) - Video: 91% authentic (likely real video) - Joint: 41% authentic (overall: FAKE)

The forensic analyst immediately knows: the audio track is the problem, not the video. They can focus their investigation on spectral analysis of the audio, speaker verification against known samples, or checking for TTS artifacts. This is actionable output that a binary score cannot provide.

The web interface (`web/templates/index.html`) displays all three scores to users, making this interpretability available to non-experts.

Purpose 2 — Architectural validation of modality specialisation:

The three-head design enables an empirical test of whether the model genuinely learns modality-specific features. If all three heads produce similar scores regardless of manipulation type, this would indicate the architecture is not achieving modality separation — the Transformer is just producing a single fused signal fed to three redundant heads.

The actual results (`draft.typ` lines 1347, 1425–1428) show the opposite — clear dissociation:

Manipulation Type	Audio Score	Video Score	Interpretation
audio_modified	Suppressed (0.375)	Elevated (0.870)	Audio head detects fake; video head sees real
visual_modified	Elevated (0.716)	Suppressed (0.700)	Video head detects fake; audio head sees real
both_modified	Suppressed (0.303)	Suppressed (0.305)	Both heads agree: fake
real	Elevated (0.716)	Elevated (0.733)	Both heads agree: real

The dissociation moves in the **correct direction** for each manipulation type. This is not a trivial result — it means the Transformer’s self-attention mechanism is learning to route modality-specific information to the appropriate head.

Purpose 3 — Multi-task learning signal:

The auxiliary losses on the audio and video heads provide additional gradient signal during training:

```
L_total = 0.4 * L_audio + 0.4 * L_video + 0.8 * L_joint
```

This multi-task objective encourages both modality encoders to produce discriminative features independently, not just features that work well when fused. A binary-output system trains only the fusion module to discriminate; the encoders may learn to produce any features as long as the fusion module can combine them. The auxiliary heads force the encoders to learn *independently useful* features.

Joint label logic: A video is labeled “real” (joint_label=1) only if BOTH audio AND video are authentic:

```
joint_label = (audio_label == 1) & (video_label == 1)
```

This means: - **real**: audio_label=1, video_label=1 → joint_label=1 - **audio_modified**: audio_label=0, video_label=1 → joint_label=0 - **visual_modified**: audio_label=1, video_label=0 → joint_label=0 - **both_modified**: audio_label=0, video_label=0 → joint_label=0

The joint head must learn to detect ANY form of manipulation, while the modality heads specialize.

4. Implementation & Engineering

Q13: Describe your training pipeline and key engineering challenges.

The pipeline is orchestrated by `main.py` → `_run_pipeline()` (lines 589–790), following five stages:

Stage 1 — Data Download (`download_data.py`): - Uses Hugging Face `datasets` library with `HF_TOKEN` environment variable for authentication. - AV-Deepfake1M++ validation split distributed as multi-part ZIP archives. - Automatic extraction via `p7zip` (`apt-get install p7zip-full`). - Validates download completeness by checking expected file count against metadata. - If data is already present (non-empty `VAL_DIR`), download is skipped.

Stage 2 — Metadata Loading & Speaker Split (`data_utils.py`): - `load_metadata()` reads `val_metadata.json` → 77,326 entries → pandas DataFrame. - Each entry: `file`, `modify_type`, `audio_frames`, `video_frames`, `fake_segments`. - `sample_videos()`: 1. Filters entries with `audio_frames > 0` AND `video_frames > 0` → 77,115 remaining. 2. Extracts speaker ID from file path: `source/id00015/clip.mp4` → `id00015`. 3. `GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=42)` on speaker groups. 4. Asserts zero speaker overlap: `assert len(overlap) == 0`. 5. Reports: modification type distribution, train/val sizes, unique speaker count.

Stage 3 — Feature Extraction (`data_utils.py` → `extract_all_features()`):

This is the most engineering-intensive stage. The function `process_split_to_disk()` (lines 427–546) implements:

- **28 CPU workers:** `multiprocessing.Pool(processes=28)` using fork-based parallelism (Linux). Each worker opens one video at a time, extracts features, saves as `.pt` file.
- **Audio extraction:** `torchaudio.load(video_path, backend="ffmpeg")` → resample to 16kHz → trim/pad to 32,000 samples → `MelSpectrogram(sr=16000, n_fft=1024, hop_length=512, n_mels=128)` → decibel conversion via `AmplitudeToDB`.
- **Video extraction:** `cv2.VideoCapture(video_path)` → read 50 frames → resize to 224×224 → BGR to RGB → normalize with ImageNet mean/std → stack to tensor (50, 3, 224, 224).
- **Manifest-based resumability:** JSON manifests track:
 - `train_manifest.json` / `val_manifest.json`: list of successfully extracted files with their `.pt` paths.
 - `train_failed.json` / `val_failed.json`: list of files that failed extraction (corrupted, missing, codec issues).
 - Progress saved every 500 successes and on any exception.
 - On restart, the script reads existing manifests and skips already-processed files.

- **File verification:** Before extraction, checks if file exists on disk. If not, records in `*_failed.json` and skips — prevents repeated attempts on missing files.
- **95% cap:** Extraction caps at 95% completion to account for inherent data corruption; this prevents infinite retry loops.

Why this matters: Vast.ai instances can terminate without notice (spot instance preemption, session expiry, payment issues). The full extraction on 68,851 clips takes approximately 6–8 hours on 28 CPU workers. Without resumable manifests, a termination at hour 7 would require restarting from zero — losing 7 hours of compute. With manifests, restart picks up from the last save point (500-file granularity), losing at most ~5 minutes of work.

Stage 4 — Training (`train_utils.py` → `train_model()`):

- **Dataloaders:** `create_dataloaders()` in `data_utils.py` creates PyTorch DataLoaders with lazy-loading datasets (`PreExtractedDataset`). Batch size 8 per GPU, `pin_memory=True`, `num_workers=4` for async loading.
- **Model initialization:** `AVDeepfakeDetector` class in `main.py:106-146` instantiates video encoder, audio encoder, and fusion module based on config.
- **Multi-GPU:** `DataParallel` wraps the model when `torch.cuda.device_count() > 1`, splitting batches across GPUs.
- **Two-phase training:** Controlled by `freeze_epochs` in `TRAIN_CONFIG`. Phase 1: `freeze_encoders(True)` → only fusion module + heads trainable. Phase 2: `freeze_encoders(False)` → all parameters trainable, encoder LR = `config['encoder_lr']` ($1e-5$).
- **Optimizer:** `AdamW(model.parameters(), lr=1e-4, weight_decay=1e-4)` — decoupled weight decay prevents regularization interference with adaptive learning rates.
- **Scheduler:** `ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=5)` — monitors validation joint AUC, halves LR when improvement stalls.
- **Gradient clipping:** `torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)` — prevents exploding gradients from 3D convolutions and Transformer attention.
- **W&B logging:** Per-epoch: `train_loss`, `val_loss`, `val_auc_audio/video/joint`, `learning_rate`, epoch time. Per-run: hyperparameters, model architecture, W&B `watch()` for gradient histograms.
- **Checkpoints** (`checkpoint_utils.py` → `CheckpointManager`): `training_checkpoint.pth` saves every epoch (for resume), `best_model.pth` saves when validation joint AUC improves. Contents: `model_state_dict`, `optimizer_state_dict`, `scheduler_state_dict`, `epoch`, `best_val_auc`, `python_rng_state`, `numpy_rng_state`, `torch_rng_state`, `cuda_rng_state`. The full RNG state ensures exact reproducibility on resume.
- **Early stopping:** Triggers when `patience` epochs (auto-computed as `max(5, round(epochs * 0.30)) = 5`) pass without validation AUC improvement.

Stage 5 — Evaluation (`main.py` → `evaluate_model()`): - Loads `best_model.pth` via `checkpoint_manager.load_best_model()`. - Multi-window inference: 3 windows per video

(start, center, end), predictions averaged. - Metrics: per-head AUC, per-type accuracy at ≥ 0.5 , confusion matrix. - Plots: audio vs video scatter (color-coded by type), joint score distribution histogram, confusion matrix heatmap. - `compare_models.py`: Multi-model comparison with reliability diagrams, per-type bar charts, calibration curves.

Key engineering challenges and solutions:

1. **Corrupted MP4 containers:** `librosa.load()` failed silently on non-standard codecs. Solution: `torchaudio.load(video_path, backend="ffmpeg")` + `MelSpectrogram` transform. The FFmpeg backend correctly extracts audio from any container FFmpeg can read (virtually all common video formats).
2. **Cloud instance termination:** Vast.ai spot instances can terminate without warning. Solution: manifest-based resumable extraction (progress saved every 500 files) + checkpoint-based resumable training (full state saved every epoch). Both systems designed for crash recovery.
3. **Checkpoint corruption (Model 1):** SCP file transfer truncated the checkpoint during download from Vast.ai instance to local machine. Solution implemented retrospectively: verify file size (`ls -la`) against expected size before terminating the Vast.ai instance. A simple check that would have saved a training run.
4. **28-worker multiprocessing:** Fork-based parallelism inherits file descriptors from the parent process. Solution: close all file handles before fork, reopen in child workers. Video files are opened and closed per-worker — no shared file state.
5. **GPU memory management on Vast.ai:** RTX 3080 has 10GB VRAM. With 50-frame 3D video tensors, `batch_size` must be 8 per GPU. `DataParallel` doubles effective batch size to 16 but requires both GPU outputs to fit in memory simultaneously. Gradient accumulation was considered but not needed at `batch_size=8`.
6. **Date persistence across Vast.ai sessions:** Vast.ai provides persistent storage volumes. Solution: mount a volume to `/workspace`, download data once, retain across sessions. The resumable pipeline handles the case where a new instance mounts the same volume — existing features and checkpoints are detected and reused.

Q14: How did you handle dataset size and storage?

Problem: AV-Deepfake1M++ full dataset is approximately 1.4 TB. Student project with no institutional storage or GPU access. Processing the full dataset was infeasible within the 5-month timeline and ~\$80 cloud GPU budget.

Three strategies:

1. Validation split only (68,851 clips): The full dataset has ~2M clips. The validation split alone has 77,326 metadata entries → 68,851 usable after filtering. This reduced raw video storage from ~1.4 TB to ~140 GB. Even this subset required careful management — downloading via Hugging Face took ~4 hours, extraction took ~2 hours.

2. Pre-extraction to disk (.pt files): Feature extraction is the most computationally expensive step (ResNet3D-18 forward passes on 50 frames × 68,851 videos = 3.4M frame inferences, plus FFT + mel conversion for audio). Extraction runs once and saves features as individual .pt files (~500 KB each for video features + audio spectrogram). Total extracted features: ~35 GB. Training uses `PreExtractedDataset` (`data_utils.py`) that lazy-loads .pt files on demand from a manifest listing. This avoids repeated video decoding and feature computation. The trade-off is disk space (~35 GB) for training speed (features are ready to load).

3. Resumable manifests: If storage fills mid-extraction or an instance terminates, the manifest-based system (`process_split_to_disk`) resumes from the last saved state. The `*_failed.json` manifest prevents repeated attempts on known-bad files (corrupted, codec issues). Extraction is incremental — workers check if a file already exists in the output directory before processing, avoiding redundant computation.

Storage footprint breakdown: | Component | Size | Notes | |———|——|——-| | Raw videos | ~140 GB | 68,851 MP4 files, various codecs | | Extracted features (.pt) | ~35 GB | One file per video: video_embedding.pt + audio_spectrogram.pt | | Checkpoints (.pth) | ~2 GB | 3 runs × ~600 MB per checkpoint folder | | Logs (W&B, output.txt) | ~1 GB | Per-epoch metrics, training console output | | **Total** | **~178 GB** | Fits within 500 GB Vast.ai SSD |

Q15: Two-phase training — detailed rationale and evidence.

Implementation (`train_utils.py`, controlled by `config.py` → `freeze_epochs`):

```
freeze_epochs = max(1, round(config['epochs'] * 0.25)) # = 2 for 10-epoch budget
```

Phase 1 details (epochs 1–2, frozen encoders): - `requires_grad = False` for all ResNet3D-18 and ResNet18 parameters. - Only the Transformer fusion module, three classification heads, and projection layers are trainable. - Optimizer: AdamW with `lr=1e-4`, `weight_decay=1e-4`. - The encoders act as fixed feature extractors — they convert raw video/audio to 256-d embeddings but do not adapt those embeddings to the detection task. - Focal Loss applied to all three heads; the fusion module learns to relate the fixed audio and video embeddings.

Why freeze first: The pretrained weights encode rich general features from Kinetics-400 (human actions — talking, gesturing) and ImageNet (1.2M images across 1,000 categories). If the randomly-initialized fusion module produces poor gradients (which it does in early epochs), unfrozen encoders would receive large, noisy gradient updates that rapidly destroy these pretrained features — catastrophic forgetting. Phase 1 protects the pretrained knowledge while the fusion module stabilizes. After ~2 epochs, the fusion module produces more coherent gradients, and it becomes safe to unfreeze the encoders.

Phase 2 details (epochs 3+, full fine-tuning): - All parameters unfrozen. - Encoder learning rate: $1e-5$ ($10\times$ lower than fusion module's $1e-4$). - Separate parameter groups in AdamW: `python optimizer = AdamW([ {'params': fusion_params, 'lr': config['learning_rate']}, {'params': encoder_params, 'lr': config['encoder_lr']}, ], weight_decay=config['weight_decay'])` # $1e-4$ - ReduceLROnPlateau halves LR when validation AUC plateaus for 5 epochs. - Early stopping after *patience* epochs (5) without improvement.

Why $10\times$ lower encoder LR: The encoders already have good weights from pretraining. Domain adaptation (Kinetics-400 action recognition $\rightarrow$ deepfake detection, ImageNet object classification $\rightarrow$ spectrogram classification) requires small adjustments, not large restructuring. A low LR allows the encoders to shift their feature representations toward the detection task while preserving the general visual/spectral knowledge they acquired during pretraining. A higher LR risks overwriting pretrained features with task-specific features that overfit to the training data.

Evidence for necessity (Model 1 baseline): - Model 1: 1 epoch Phase-1-only training (frozen encoders, fusion module + heads only). Validation AUC: 0.663. - All Phase-2-trained models: validation AUC 0.985, test AUC 0.915. - The ~0.32 AUC gap demonstrates that frozen encoders alone cannot solve the detection task — cross-modal representations require encoder adaptation to the target domain. - Model 1's prediction pattern (all outputs 0.23–0.35, always predicting FAKE) indicates an underfit classifier — the fusion module never converged because the fixed encoder features were insufficiently discriminative for the detection task.

Q16: Data loading, augmentation, and multi-window inference.

Lazy-loading Dataset (data_utils.py $\rightarrow$ PreExtractedDataset):

```
class PreExtractedDataset(Dataset):
    def __init__(self, features_dir, manifest_path):
        with open(manifest_path) as f:
            self.manifest = json.load(f) # List of {file, path, type}
            self.features_dir = features_dir
```

```
def __getitem__(self, idx):
    entry = self.manifest[idx]
    video_feat = torch.load(os.path.join(self.features_dir, entry['video_path']))
    audio_feat = torch.load(os.path.join(self.features_dir, entry['audio_path']))
    return {'video': video_feat, 'audio': audio_feat, ...}
```

Features are pre-extracted `.pt` files loaded on demand. No video decoding or FFT computation during training — pure tensor I/O. This enables high throughput: ~30 batches/second on dual RTX 3080.

Audio augmentations (SpecAugment, training only): - Frequency masking: Randomly masks up to 20 consecutive mel frequency bins. Simulates frequency-selective noise, channel dropout, or band-limited recording conditions. - Time masking: Randomly masks up to 15 consecutive time steps (~0.5 seconds). Simulates temporal dropout, audio gaps, or intermittent interference. - Implementation: `torchaudio.transforms.FrequencyMasking(freq_mask_param=20)` and `TimeMasking(time_mask_param=15)` applied sequentially to the mel-spectrogram before the ResNet18 encoder. - Reference: Park et al. (2019), “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”, Interspeech.

Video augmentations (training only): - Random horizontal flip ($p=0.5$): Doubles effective training data; face orientation is symmetric for deepfake detection purposes. - Brightness jitter (± 0.2): Simulates variable lighting conditions common in real-world video. - Contrast jitter (0.8–1.2): Simulates camera exposure variations and video compression. - Applied per-frame independently via `torchvision.transforms.ColorJitter`.

No augmentations for validation or inference — the model sees clean, unmodified features to ensure evaluation reflects true performance.

Multi-window inference (`data_utils.py` → `extract_multiple_windows()`): - During evaluation, 3 different 2-second windows are extracted from each video: start (first 2s), center (middle 2s), end (last 2s). - Each window is processed independently through the model. Predictions from all 3 windows are averaged. - This improves robustness: a manipulation that occurs only in the middle of a video is captured by the center window, while start/end windows provide context. - For fake videos with `fake_segments` annotations, windows are preferentially sampled near the annotated manipulation regions. - Trade-off: $3\times$ inference cost for improved robustness. At inference time (~100ms per window on GPU), this is acceptable.

Q17: Web interface and inference system details.

CLI (inference.py, 540 lines standalone): - No training pipeline dependencies — re-defines model architectures inline (AVDeepfakeDetector, TransformerFusion, ResNet3D-18, ResNet18). - Arguments: `--model` (path to .pth), `--video` (single file), `--video_dir` (batch), `--output` (CSV path), `--device` (cpu/cuda), `--n_windows` (default 3). - Workflow: load model → extract 3 windows → average predictions → output verdict + per-head scores. - Device-agnostic: `torch.load(checkpoint, map_location=device)` handles CPU/GPU mapping.

Web Interface (web/app.py, Flask, 276 lines):

Backend architecture: - 5 REST API endpoints: `GET /api/models`, `POST /api/analyze`, `POST /api/compare`, `GET /api/history`, `DELETE /api/history/<id>`, `DELETE /api/history`. - Model caching: `_model_cache` dictionary stores loaded models — each model is loaded once at first request, shared across subsequent requests. - Temporary file handling: Uploaded videos saved to `tempfile.gettempdir()` with UUID prefix, processed, immediately deleted in `finally` block. - SQLite history: `init_db()` creates table if not exists with schema (id, filename, model_key, joint_score, audio_score, video_score, confidence, threshold, verdict, timestamp). Schema migration: drops and recreates table if column count doesn't match `EXPECTED_COLS = 10`. - 500MB upload limit via `app.config["MAX_CONTENT_LENGTH"]`. - Supported formats: mp4, avi, mov, mkv, webm, m4v. - Error handling: custom handlers for 413 (too large), 404 (not found), 500 (internal error), and unhandled Exception — all return JSON.

Model paths (web/app.py:28-29):

```
MODEL1_PATH = "logs/logs_2/best_model.pth" # Model 2 (epoch 5, AUC 0.994)
MODEL2_PATH = "logs/logs_3/best_model.pth" # Model 3 (epoch 3, AUC 0.985)
```

Environment variable overrides: `MODEL1_PATH`, `MODEL2_PATH`, `DB_PATH`, `MAX_UPLOAD_MB`, `PORT`.

Frontend (web/templates/index.html, single-page vanilla JS): - **Analyze tab:** Drag-and-drop upload area, model selector dropdown (“Model 2 — Val AUC 0.994” / “Model 3 — Test AUC 0.937”), threshold slider (default 0.5). Submit sends `POST /api/analyze` with `FormData` (model key, threshold, video file). Response displays verdict (“FAKE” in red / “REAL” in green), three per-head probability scores, confidence level. The screenshot `web_analyze_fake.png` shows a fake audio_modified clip: video score 0.91 (genuine video), audio score 0.23 (synthesized audio), joint score 0.41 → FAKE verdict. This illustrates the modality dissociation behavior. - **Compare tab:** Same upload + two model indicators. Sends `POST /api/compare`. Side-by-side display with agree/disagree badge. The screenshot `web_compare.png` shows both models agreeing on a fake verdict with slightly different score distributions (Model 2: narrower margins, Model 3: wider separation). - **History tab:** Displays `GET /api/history` as a sortable table.

Per-entry delete (DELETE /api/history/<id>), bulk clear (DELETE /api/history). Useful for auditing past analyses.

5. Training & Experiments

Q18: Experimental setup and four training runs.

Four training sessions conducted across three configurations:

Run	Epochs	Infrastructure	Val AUC	Phase	Status	Notes
Model 1	1	Google Colab (free)	0.663	Phase 1 only	Corrupted download	Frozen encoders; underfit
Model 2	5	Vast.ai (2× RTX 3080)	0.994	Phase 1 (2ep) + 2 (3ep)	Intact	Highest val AUC
Model 3	3	Vast.ai (2× RTX 3080)	0.985	Phase 1 (2ep) + 2 (1ep)	Intact	Early stop; best test perf.
Model 4	5	Vast.ai (2× RTX 3080)	0.993	Phase 1 (2ep) + 2 (3ep)	Intact	Same session as M3

Session relationships: - Models 1, 2: Independent sessions (separate random initialization, separate feature extraction, separate training runs). - Models 3 and 4: Same session — Model 3 is the epoch-3 checkpoint, Model 4 is the epoch-5 checkpoint. Epochs 1–3 are shared. This within-session pairing is crucial for analysing the effect of extended training on score calibration.

Training configuration (all runs, from config.py): - Optimizer: AdamW with decoupled weight decay (1e-4). Fusion module LR: 1e-4. Encoder LR: 1e-5 (10× lower). Weight decay: 1e-4. - Loss: Focal Loss ($\gamma=2.0$, $\alpha=0.25$). Combined: $L = 0.4 \times L_{\text{audio}} + 0.4 \times L_{\text{video}} + 0.8 \times L_{\text{joint}}$. - Scheduler: ReduceLROnPlateau (mode='max', factor=0.5, patience=5). Monitors validation joint AUC. - Batch size: 8 per GPU × 2 GPUs (DataParallel) = 16 effective. - Gradient clipping: max norm 1.0. - Freeze epochs: 2 (Phase 1). - Patience (early stopping): 5 epochs. - Seed: 42 (Python, NumPy, PyTorch — full reproducibility). - Architecture: Identical across all runs (ResNet3D-18 + ResNet18 + TransformerFusion). - Dataset: Same 68,851-clip validation split, same speaker-disjoint partition.

Resource constraints: - All runs capped at 5 epochs (default 10-epoch budget in `config.py` never exercised). - Cloud GPU cost: ~\$0.30–0.50/hr per Vast.ai instance (2× RTX 3080). Training time: ~6–8 hours per run. Approximate total cost: \$60–80. - No hyperparameter sweep feasible — all runs used identical hyperparameters. W&B Bayesian sweep configuration (SWEEP_CONFIG in `main.py:221–233`) was prepared but never executed. - Model 1’s single epoch on Colab free tier (Tesla T4, limited to ~4 hours session, no persistent storage).

Hyperparameter justification: - `focal_gamma=2.0`: Standard from Focal Loss paper (Lin et al., 2018). `=0` reduces to BCE; `=2` provides quadratic down-weighting of easy examples. - `focal_alpha=0.25`: Provides class rebalancing favoring the minority class (real videos). - `dropout=0.4`: Higher than typical 0.2–0.3 because dataset is large and generalization is the primary concern. - `feature_dim=256`: Balanced between expressiveness and overfitting risk. Half the ResNet output dimension (512). - `hidden_dim=512 = 2×feature_dim`: Standard practice for fusion layers — gives sufficient capacity without being excessive. - `batch_size=8` per GPU: Largest that fits in 10 GB VRAM with 50-frame 3D video inputs.

Q19: Why did Model 1 fail?

Observed behaviour: Validation AUC 0.663 after 1 epoch. Prediction pattern: all outputs in 0.23–0.35 range, always predicting FAKE regardless of ground truth.

Root causes:

- 1. Insufficient training (1 epoch Phase 1 only):** Phase 1 freezes both encoder backbones. Only the randomly-initialized Transformer fusion module (4.7M params) and three classification heads (1.5K params) are trainable. One epoch (~2,000 batches at `batch_size=8` on ~55,000 training samples after 80/20 split on 68,851) is insufficient for the fusion module to converge from random initialization. The module needs to learn to:
 - Project 256-d embeddings to 512-d (linear layer)
 - Attend across the [CLS, video, audio] sequence (2-layer Transformer, 8 heads)
 - Produce discriminative CLS representations for the classification heads This is ~4.7M parameters that need meaningful weight updates from a single pass through the data.
- 2. Underfit classifier predicting majority class:** The 0.23–0.35 score range indicates the sigmoid outputs are compressed near the random-initialization midpoint (~0.5 for randomly initialized Linear layers). The classifier never learned to separate real from fake — it defaults to the majority class (fake videos outnumber real in a 3:1 ratio across the 4-type split). All outputs below 0.5 → always predicting FAKE.

3. **Checkpoint corruption:** The `.pth` file was corrupted during SCP download from Colab to local machine. File transfer was truncated (partial download). This prevented post-hoc analysis — the exact weight state at epoch 1 is unknown, and the model cannot be loaded for inference on test data. A simple `ls -la` file-size check before terminating the Colab session would have caught the truncated transfer.

Lessons: - Phase 1 should be at least 2 epochs (which is what the auto-computed `freeze_epochs = max(1, round(epochs×0.25))` produces for epochs 8, but for epochs=5, this rounds to 1–2). - Verify checkpoint file size against expected size (~600 MB for the full model) before terminating cloud instances. - One epoch is insufficient for any phase — the training loss curve should show convergence before proceeding.

Value as baseline: Despite the failure, Model 1 serves as a valuable baseline demonstrating that Phase 2 fine-tuning is essential. The ~0.32 AUC gap between Model 1 (0.663) and Phase-2-trained models (0.985) quantifies the contribution of encoder fine-tuning — one of the few near-ablation comparisons available in this project.

Q20: Hyperparameter selection approach.

No systematic hyperparameter tuning was conducted. This is a significant limitation (dissertation Section 5.5).

Approach taken: - All hyperparameters set based on standard practice in the literature and initial trial runs on small data subsets. - `freeze_epochs` and `patience` auto-computed as fractions of the total epoch budget: - `freeze_epochs = max(1, round(epochs × 0.25))` — Phase 1 duration. - `patience = max(5, round(epochs × 0.30))` — early stopping patience. - All runs used identical hyperparameters — no variant tested.

Prepared but not executed: W&B Bayesian sweep configuration (`main.py:221–233`):

```
SWEEP_CONFIG = {
    'method': 'bayes',
    'metric': {'name': 'val/auc_joint', 'goal': 'maximize'},
    'parameters': {
        'focal_gamma': {'values': [0.5, 1.0, 2.0, 3.0]},
        'focal_alpha': {'values': [0.1, 0.25, 0.5, 0.75]},
        'dropout': {'min': 0.2, 'max': 0.5},
        'learning_rate': {'min': 1e-5, 'max': 1e-3, 'distribution': 'log_uniform_values'},
        'hidden_dim': {'values': [256, 512, 1024]},
        'freeze_epochs': {'values': [4, 8, 12]},
        'weight_decay': {'min': 1e-5, 'max': 1e-3, 'distribution': 'log_uniform_values'},
```

```
}
}
```

Bayesian optimization was chosen over grid search because it intelligently samples promising regions of the hyperparameter space based on previous trial results, requiring fewer trials for comparable coverage. With 20 planned trials (`--sweep_count 20`), the sweep would have provided meaningful hyperparameter sensitivity analysis. Not run due to: each trial requires a full training run (~6–8 hours), 20 trials would cost approximately \$120–200, exceeding the cloud GPU budget.

Justification for chosen values: - `focal_gamma=2.0`: Original Focal Loss paper recommendation for object detection with severe class imbalance. The 4-type split has a 3:1 fake-to-real ratio. - `focal_alpha=0.25`: Balances toward the minority (real) class. - `dropout=0.4`: Higher than standard 0.2–0.3 to prevent overfitting given the ~55K training samples and 49.7M parameters. - `learning_rate=1e-4`: Standard for AdamW on Transformer architectures. Log-uniform sweep range [1e-5, 1e-3] would have explored an order of magnitude in each direction. - `encoder_lr=1e-5`: 10× lower than fusion LR — standard transfer learning practice.

What I would do with more budget: Run the W&B Bayesian sweep with 20 trials, each trained for 5 epochs. Analyse hyperparameter importance via W&B parallel coordinates plots. Select best hyperparameters and run final training for 10+ epochs with ReduceLROnPlateau full cycle.

6. Results & Evaluation

Q21: Model 2 vs Model 3 — detailed analysis.

This is the central empirical finding of the project. Complete comparison:

Metric	Model 2 (epoch 5)	Model 3 (epoch 3)	Gap	Winner
Val Joint AUC	0.994	0.985	+0.009	M2
Test AUC	0.919	0.937	−0.018	M3
Accuracy (=0.5)	66.0%	93.0%	−27pp	M3
Precision	—	1.000	—	M3
Recall	—	0.720	—	M3
F1	0.575	0.837	−0.262	M3
False positives	—	0	—	M3
False negatives	—	7	—	M3
Best threshold	0.795	0.533	—	Application-dependent

Metric	Model 2 (epoch 5)	Model 3 (epoch 3)	Gap	Winner
Acc at best	87.0%	93.0%	−6pp	M3
F1 at best	0.764	0.837	−0.073	M3
Calibration gap	0.31	0.19	+0.12	M3
Per-type (audio_mod)	48%	100%	−52pp	M3
Per-type (visual_mod)	60%	100%	−40pp	M3
Per-type (both_mod)	68%	100%	−32pp	M3
Per-type (real)	92%	72%	+20pp	M2

Model 3 wins 5 of 7 head-to-head metrics (AUC, accuracy, F1, precision, FP) and all three fake-type categories.

The mechanism — score compression:

Model 2 (5 epochs of fine-tuning) produces scores that are well-ranked (0.994 AUC means it correctly orders 99.4% of real/fake pairs) but compressed toward 0.5:

- Real videos: scores 0.85–0.97 (mean ~ 0.733).
- Fake videos: scores 0.30–0.45 (mean ~ 0.355).
- Gap between means: ~ 0.378 .

Model 3 (1 epoch of fine-tuning, saved at the first fine-tune checkpoint) produces wider score separation:

- Real videos: scores 0.30–0.97 (mean ~ 0.676 , wider spread with some lower real scores).
- Fake videos: scores 0.05–0.30 (mean ~ 0.146 , much lower than Model 2).
- Gap between means: ~ 0.530 .

With wider separation, the $=0.5$ threshold cleanly divides the distributions. Model 2’s narrower gap means many videos land near 0.5, where small score perturbations flip the classification.

Why this happens: Extended fine-tuning (epochs 3–5) causes score compression — a form of mild overfitting where the model continues to optimize the training objective (ranking) at the expense of calibration quality. This is consistent with Guo et al. (2017) who observed that modern neural networks become less calibrated with longer training even as accuracy improves.

At optimal thresholds: The performance gap narrows significantly. Model 2 at $\theta=0.795$ achieves 87% accuracy (F1 0.764). Model 3 at $\theta=0.533$ achieves 93% accuracy (F1 0.837). The remaining 6pp gap is attributable to Model 3’s wider inherent score separation — even after threshold optimization, its score distributions are more separable.

Application-specific trade-off: - If false positives are costly (public content moderation, legal evidence): Model 3 with zero false positives. - If false negatives are costly (forensic triage,

missing a deepfake matters): Model 2 with lower false negative rate (8% vs 28%). - There is no universally optimal model — the choice depends on application-specific error costs.

Caveat: This observation is based on n=3 loadable checkpoints from 5-epoch training runs. It is hypothesis-generating, not confirmatory. Whether extended training (20+ epochs) with full ReduceLROnPlateau cycles would reverse, persist, or stabilize this pattern is unknown — this requires future work with longer training budgets.

Q22: Per-type accuracy — why `audio_modified` is hardest.

Model 3 per-type joint accuracy at $\tau=0.5$ (from `draft.typ` lines 1299–1324):

Type	Accuracy	Mean Audio Score	Mean Video Score	Mean Joint Score	Notes
real (n=25)	72%	0.716	0.733	0.676	7 false negatives
audio_modified (n=25)	100%	0.375	0.870	0.409	All correctly detected
visual_modified (n=25)	100%	0.716	0.300	0.304	All correctly detected
both_modified (n=25)	100%	0.303	0.305	0.146	All correctly detected

Note: Model 3 achieves 100% on all fake categories but only 72% on real (7/25 real videos misclassified as fake). This is the precision-recall trade-off: aggressive fake detection catches all fakes but flags some real videos.

Why `audio_modified` scores are closest to the boundary (across all models):

The three-head architecture produces independent scores per modality. For `audio_modified`:

1. The **video encoder** processes 50 genuine, unmodified frames. ResNet3D-18 finds no temporal artifacts — lip motion is natural, no frame-to-frame jitter. Output: high video score (0.870 for Model 3).

2. The **audio encoder** processes a mel-spectrogram from TTS/VC-synthesized speech. ResNet18 detects spectral artifacts — over-smoothed formants, unnatural harmonic structure, missing high-frequency fricative energy. Output: suppressed audio score (0.375 for Model 3).
3. The **fusion module** receives conflicting signals: [CLS attends to video_feat = “looks real”] vs [CLS attends to audio_feat = “sounds fake”]. The [CLS] token must reconcile these. Output: intermediate joint score (0.409).
4. This joint score is the *highest* fake score across all manipulation types (0.409 vs 0.304 for visual, 0.146 for both). The genuine video signal pulls the joint prediction toward 0.5 — the decision boundary.

Why visual_modified is easier: - Genuine audio → high audio score. - Synthesized video → suppressed video score (ResNet3D-18 detects lip motion artifacts). - However: the ResNet3D-18 is a stronger discriminator than the ResNet18 (visible in per-head AUC curves in `training_history.png` — video AUC consistently higher than audio AUC). The video head provides a stronger fake signal than the audio head can counteract.

Why both_modified is easiest: - Both modality heads provide consistent fake evidence → joint score is pulled strongly toward 0.0. - No conflicting signals to reconcile → clean, confident fake prediction.

Implication for future work: Improving the audio encoder is the highest-impact architectural change. The mel-spectrogram + ResNet18 approach detects some TTS/VC artifacts but produces weaker suppression than the video encoder. A better audio encoder (Wav2Vec 2.0 on clean audio, or a dedicated audio deepfake detection model like RawNet2) could close the audio_modified gap.

Q23: Calibration curves — detailed interpretation.

Reliability diagrams (`figures/calibration_curves.png`, generated by `plot_calibration_curves.py`):

How to read: - X-axis: Predicted probability, binned into 10 deciles ([0–0.1], [0.1–0.2], ..., [0.9–1.0]). - Y-axis: Observed accuracy — fraction of samples in that bin that were actually “real” (ground truth). - Perfect calibration = points lie on the diagonal. A model predicting 0.80 should be correct 80% of the time. - Above diagonal = overconfident (model predicts higher probabilities than justified). - Below diagonal = underconfident (model predicts lower probabilities than justified).

Model 3 (best calibration): - Closely follows the diagonal across all 10 bins. Calibration gap (mean absolute deviation from diagonal) = 0.19. - Bin [0.9–1.0]: Observed accuracy 0.95 (close to expected ~0.95). The model is slightly overconfident at high probabilities. - Bin [0.0–0.1]:

Observed accuracy = 0.05 (close to expected ~0.05). Well-calibrated at low probabilities. - Middle bins [0.4–0.6]: Very few samples (wide score separation means few videos land in this range). Good calibration of the handful that do.

Model 2 (worst calibration): - Significant deviation from the diagonal, especially in mid-range. Calibration gap = 0.31. - Bin [0.4–0.6]: Many samples land here (compressed scores). Observed accuracy = 0.35–0.60 instead of expected 0.45–0.55. The model is uncertain but also unreliable in its uncertainty region — the worst combination. - High-probability bins [0.7–0.9]: Observed accuracy = 0.60–0.75, well below the diagonal. The model is overconfident — predicting 0.80 but correct only ~65% of the time. - This explains the 66% accuracy at $\tau=0.5$: many samples cluster near 0.5 with unreliable calibration.

Model 4 (intermediate calibration): - Moderate deviation from diagonal. Calibration gap = 0.21 (close to Model 3 but with more samples in the uncertain mid-range). - Performance at $\tau=0.5$: 71% — better than Model 2 (66%) but worse than Model 3 (93%) because more samples populate the 0.4–0.6 range.

Practical significance: - For deployment with a fixed threshold, calibration quality directly determines accuracy. A model with AUC 0.994 but poor calibration is unusable at standard thresholds; a model with AUC 0.985 and good calibration is deployable. - Threshold optimization can recover much of the ranking quality (Model 2 at $\tau=0.795 \rightarrow 87\%$ accuracy), but this requires a held-out calibration set with known ground truth — not always available in real deployment scenarios. - The field should report calibration metrics alongside AUC and accuracy. Reliability diagrams require one extra plot per model — negligible cost, substantial additional insight.

Methodological contribution: Calibration analysis is rarely reported in prior multimodal deepfake detection work. Its inclusion here demonstrates a more complete evaluation methodology that considers deployment readiness, not just ranking performance.

Q24: Statistical reliability of results.

Test set composition: 100 videos — 25 per manipulation type (real, audio_modified, visual_modified, both_modified). Sampled from the validation split using `create_test_data.py` with a second GroupShuffleSplit producing 21 unique test speakers with zero overlap with training speakers.

Wilson 95% confidence intervals:

Metric	Observed	95% CI Lower	95% CI Upper
Overall accuracy (M3)	93.0%	86.3%	96.6%
Per-type accuracy (fake, n=25)	100.0%	86.7%	100.0%
Per-type accuracy (real, n=25)	72.0%	52.4%	85.7%
False positive rate	0.0%	0.0%	11.3% (rule of three)

What these CIs mean: - Overall accuracy of 93% could be anywhere from ~86% to ~97% at 95% confidence. This is a 11-percentage-point range. - Per-type accuracy of 100% (on 25 fake

videos each) could be as low as ~87%. A single misclassification would change it to 96% — a 4pp shift. - Zero false positives on 25 real videos: the true FPR upper bound is ~11.3% (using the rule of three: 3/25). If the model had one false positive, FPR would jump to 4%. The “zero false positives” claim is fragile to small sample size.

What can be concluded: - The model clearly outperforms random chance (AUC 0.937, accuracy 93%, both well above 50% baseline). - Model 3 > Model 4 by accuracy (93% vs 71%) is a 22pp difference — likely real even with wide CIs. - Model 3 > Model 2 by accuracy (93% vs 66%) is a 27pp difference — same conclusion. - The pattern of per-type difficulty (real easiest → visual/both medium → audio hardest) is consistent but not statistically confirmed per type.

What cannot be concluded: - The exact accuracy level ($93\% \pm 5.3\text{pp}$ at 95% CI). - That Model 3 is definitively the best for all possible held-out speakers. - That zero false positives will hold on a larger deployment set. - That the per-type ranking generalizes beyond AV-Deepfake1M++.

Mitigation: All conclusions are qualified with these limitations (dissertation Section 5.5, 6.2). Results are presented as observations from a small experiment, not as established findings. A test set of 500 videos per manipulation type (2,000 total) would be needed for reliable estimates, as recommended by Dolhansky et al. (2020).

Q25: Metrics — choices and limitations.

Metric	What it measures	Why used	Limitation
AUC	Ranking ability (threshold-independent). Probability that a randomly chosen real video scores higher than a randomly chosen fake video.	Standard in detection literature (Rossler, Dolhansky, Cai). Handles class imbalance — not affected by the 3:1 fake-to-real ratio in the 4-type split.	Says nothing about threshold performance. Model 2: AUC 0.994 but accuracy 66%. AUC can be high while all scores cluster near 0.5. Should be supplemented with calibration.

Metric	What it measures	Why used	Limitation
Accuracy (=0.5)	Fraction of correct classifications at the default threshold.	Intuitive, practical. Matches the deployment scenario where a fixed threshold is used.	Highly dependent on choice. 0.5 is arbitrary — no reason it’s optimal. The 27pp gap between M2 and M3 at =0.5 narrows to 6pp at optimal thresholds. Should report accuracy at optimal alongside default .
Precision	TP / (TP + FP). Of videos predicted fake, what fraction are truly fake.	Critical for low-FP applications (content moderation, legal). Model 3’s precision 1.000 means every fake prediction was correct — zero false accusations.	Doesn’t measure recall. A model predicting “real” for everything has undefined precision but no utility. Must be paired with recall.
F1 Score	$2 \times (P \times R) / (P + R)$. Harmonic mean of precision and recall.	Balances FP and FN. Single number for model comparison.	Assumes equal cost for FP and FN, which is rarely true. A model with precision 1.0 and recall 0.5 gets F1 0.667; a model with both 0.8 gets F1 0.8. The F1 doesn’t capture the qualitative difference between “never wrong when calling fake” and “occasionally wrong in both directions.”

Metric	What it measures	Why used	Limitation
Calibration gap	Mean absolute deviation from diagonal on reliability diagram.	Measures score quality independent of threshold. Directly assesses deployment readiness.	Sensitive to binning strategy (number of bins, bin boundaries). Small-sample bins (few videos per decile with $n=100$) have high variance. Single-number summary loses the bin-by-bin detail of the reliability plot.
Confidence intervals (Wilson)	Quantifies uncertainty from small test set.	Honesty about limitations. Prevents overinterpretation of small-sample results.	Only captures sampling variance, not model variance (would need bootstrapping) or dataset bias.

What I should have used but didn't: - **t-DCF** (tandem Detection Cost Function): Standard in speaker verification (ASVspoof). Models application-specific costs — assigns different weights to missed detections vs false alarms. More deployment-relevant than F1 because costs are explicit and configurable. - **ECE** (Expected Calibration Error): Weighted average of calibration gap across bins, weighted by bin sample count. Single-number calibration summary that complements reliability diagrams.

7. Critical Analysis & Tough Questions

Q26: You trained on the validation split and tested on 100 videos. Why trust these results?

The honest answer: These results should be treated as preliminary observations from a resource-constrained student project, not as established findings for publication. However, they have internal validity:

Mitigating factors:

1. **Rigorous evaluation protocol despite small scale:** The speaker-disjoint partition at two levels (train/val split + independent test set split) is a stricter evaluation than random-split protocols common in the literature (Rossler et al., 2019). The 100-video test set has zero speaker overlap with training — at either the main split level or the test set creation level. The reported metrics reflect detection performance on entirely unseen identities, which is a meaningful measure even with small sample size.
2. **Internal consistency across multiple dimensions:** The finding that extended training degrades calibration ($M3 > M4$ despite same session) is internally consistent. The per-type score dissociation (audio head suppressed for audio_modified, video head suppressed for visual_modified) is qualitatively correct — it matches the architectural design intent. Multiple measurements (AUC, accuracy, precision, calibration curves) point in consistent directions.
3. **Calibration analysis adds methodological rigor:** Reliability diagrams reveal *why* Model 2 underperforms despite higher AUC. This is not just a performance claim — it’s a mechanistic explanation supported by visual evidence.

What I would do with more resources (Section 6.4): 1. Train on the full AV-Deepfake1M++ training split (1M+ clips). 2. Evaluate on the official challenge test set (thousands of videos). 3. Run 5-fold cross-validation with different speaker partitions. 4. Report confidence intervals on all metrics. 5. Test on at least one held-out cross-dataset benchmark (FakeAVCeleb, DFDC).

Q27: Without ablations, how do you know what contributed?

I don’t. This is the most significant limitation (dissertation Section 5.5).

What I can attribute: - **Phase 2 fine-tuning is essential:** The natural ablation exists — Model 1 (Phase 1 only, AUC 0.663) vs all Phase-2 models (0.985). This ~0.32 AUC gap confirms that encoder adaptation is necessary. - **The three-head architecture achieves modality specialisation:** The dissociation pattern is empirical evidence — scores dissociate by type in the predicted direction. This is an architectural validation, not a component comparison.

What I cannot attribute: - Focal Loss vs BCE contribution: No comparison run with BCE under identical conditions. - Transformer vs MLP fusion contribution: No MLP fusion variant trained. These exist in the codebase (`PretrainedFusion`, `SimpleFusion` in `cross_modal.py`) but were never trained on the full dataset. - ResNet3D-18 vs 2D CNN: No 2D variant trained. - Full-frame vs lip-region crops: Only full-frame encoding used.

Why ablations weren’t done: Each ablation requires a full training run (~6–8 hours, \$15–25). A comprehensive set (3 components $\times$ 2 variants = 6 extra runs) would cost \$90–150

and ~36–48 hours of compute time. This exceeded the cloud GPU budget (~\$80 total) and timeline. The dissertation explicitly identifies this as the highest-priority future work and does not claim attribution beyond Phase 2.

Q28: Your test set is 100 videos from the same dataset. Could results be optimistic?

Yes, for two reasons:

1. **Test set size (statistical reliability):** $n=100$ (25 per type). Single misclassification = 1pp overall, 4pp per type. 95% CIs are wide. This is a precision problem, not a bias problem — the estimates are honest but imprecise.
2. **Same-distribution evaluation (generalization validity):** Training and testing on different splits of the same dataset — even with speaker-disjoint partitioning — doesn’t test generalization to different:
 - Generation pipelines (different TTS/VC systems, different NeRF implementations)
 - Recording conditions (different cameras, lighting, backgrounds)
 - Compression codecs (H.264 vs H.265 vs AV1)
 - Resolution and frame rates
 - Audio recording quality (microphone types, room acoustics)

The speaker-disjoint split addresses *identity* overfitting but not *dataset* overfitting. Deepfake detectors are known to learn dataset-specific artifacts (Dolhansky et al., 2020). AV-Deepfake1M++ was created by a specific set of generation tools applied to a specific set of speakers recorded under specific conditions. The model may have learned to detect artifacts of those specific tools rather than general manipulation patterns.

Honest interpretation: “Model 3 achieves 93% accuracy at detecting manipulations produced by the AV-Deepfake1M++ generation pipeline, on speakers not seen during training” — NOT “Model 3 detects any deepfake from any source with 93% accuracy.”

Q29: “Zero false positives” — lower bound analysis.

With 25 real test videos and zero observed false positives: - Observed FPR = $0/25 = 0\%$. - 95% CI upper bound via rule of three: $3/25 = 12\%$. - Exact binomial 95% CI: $[0\%, 13.7\%]$. - If one real video scored below 0.5 $\rightarrow$ FPR = 4%.

What this means: The true false positive rate could be anywhere from 0% to ~14%. At 14% FPR, the model would incorrectly flag ~14% of genuine videos as fake — unacceptable for most deployment scenarios. “Zero false positives” is a statement about what was observed on 25 videos, not about what would be observed on a larger deployment set.

How to establish FPR with confidence: To bound the true FPR below 5% at 95% confidence, you need to observe zero false positives on at least 59 real videos (rule of three: $3/59 = 5.1\%$). To bound below 1%, you’d need at least 299 real videos. This is clearly documented in the dissertation limitations.

Q30: How interpretable are the three-head scores to non-experts?

What the three-head output provides: Modality-level interpretability — tells you *which modality* triggered detection. Example output: Audio 23% authentic | Video 91% authentic | Joint 41% authentic $\rightarrow$ FAKE. The user knows: “the audio is the problem, the video looks real.”

What it does not provide: 1. **Spatial localization:** Which part of the video frame triggered detection? Attention maps over frames, lip region heatmaps — not implemented. 2. **Temporal localization:** When in the video does manipulation start/end? Requires sliding window or frame-level architecture — the `fake_segments` annotations exist but aren’t used for prediction. 3. **Feature-level explanation:** What specific artifact (spectral irregularity at 4kHz, motion jitter at frame 23) was detected? Requires feature attribution methods (SHAP, LIME, Integrated Gradients) — not implemented.

Where this sits on the interpretability spectrum: - Below: Binary output (“87% fake”) — no insight into why. - Current: Modality-level output (“audio is fake, video is real”) — useful for forensic triage. - Above (future work): Attention maps, temporal boundaries, feature attribution — true explainable AI.

Assessment: The three-head architecture is a meaningful first step toward interpretability, particularly for forensic analysts who need to know which modality to investigate. But it should not be confused with full explainable AI. The web interface makes this modality-level information accessible to non-experts.

Q31: If you did this project again, what would you change?

Five changes, in order of impact:

1. **Secure institutional GPU access before starting.** This was the binding constraint. Cloud GPU costs (\$60–80) limited training to 5 epochs across 4 runs, prevented hyperparameter sweeps, and prevented ablation studies. With free institutional GPUs, I would run: (a) 10–20 epoch training on the full dataset, (b) W&B Bayesian sweep over 7 hyperparameters, (c) 6+ ablation runs (Focal Loss/BCE, Transformer/MLP, 3D/2D CNN, full-frame/lip-region), and (d) cross-dataset evaluation on 2–3 external benchmarks. The dollar cost was modest but real as a student constraint.
2. **Run small-scale ablations early, not after full training.** Instead of training the complete architecture on the full 68K-clip dataset and discovering calibration issues at evaluation time, I would: train 3 variants (Focal/BCE, Transformer/MLP, 3D/2D) on 5,000-clip subsets for 5 epochs each. This takes ~30 minutes per run on 2 GPUs and would provide preliminary evidence for design decisions within the first month.
3. **Verify file integrity before terminating cloud instances.** Model 1’s checkpoint was corrupted during SCP download. A `stat` call checking file size (~600 MB expected for the full model) against the remote file would have caught the truncated transfer. This cost an entire training run’s checkpoint and prevented even basic analysis.
4. **Use a subset of the full training split, not just the validation split.** The Hugging Face dataset supports partial downloads by file pattern. Sampling 200K–300K clips from the training split (instead of using the 68K validation split) would provide 3–5× more training data with greater speaker and generator diversity, fitting within the same 500 GB storage budget.
5. **Report statistical confidence from the start.** Wilson CIs for accuracy, DeLong CIs for AUC. These are available in `statsmodels` and `scipy` and take ~5 lines of code. Reporting 93% [86.3%, 96.6%] from the beginning prevents overinterpretation and establishes methodological rigor.

8. Limitations & Future Work

Q32: Three most important limitations.

1. **No ablation studies (most critical):** Cannot attribute performance to any individual architectural component. The model achieves 93% accuracy but I cannot say whether Focal Loss, Transformer fusion, ResNet3D-18, or two-phase training individually contributed, or

whether a simpler variant would match performance. This is the foundation for all architectural claims — without it, I can report results but not explain them.

2. Small, same-distribution test set: 100 videos from the same dataset split. Combines (a) statistical unreliability (wide CIs, ± 5 pp on accuracy) with (b) unknown generalization to different generators and recording conditions. The zero false positives claim is particularly fragile — the 95% CI upper bound for FPR is $\sim 14\%$.

3. 5-epoch training cap: Models 2 and 4 halted while validation AUC was improving. Unknown whether extended training improves calibration, changes model ranking, or stabilizes performance. The $M3 > M2$ finding may be an artifact of limited training — it might reverse, persist, or stabilize with 20+ epochs. Hyperparameter sensitivity is completely unexplored.

Other significant limitations: Single dataset, fixed 2-second window, no temporal localization, validation-split-only training (68K vs 1M+ clips), no adversarial robustness testing.

Q33: Top three future work priorities.

1. Ablation studies (highest priority, foundation for all other work): - Focal Loss vs BCE: Same architecture, same data, same seed. Isolate loss function contribution. - Transformer vs MLP fusion: `TransformerFusion` vs `PretrainedFusion` from `cross_modal.py`. Both already implemented — just needs training. - Full-frame vs lip-region crops: Does focusing on the lip region improve detection? - Drop-one-modality: Train with audio only, video only, both — quantify independent modality contributions. - Estimated cost with institutional GPUs: 4 variants $\times$ 5 epochs $\times$ 6 hours = ~ 120 GPU-hours. Achievable in 1–2 weeks.

2. Cross-dataset evaluation (most practically relevant): - Test on FakeAVCeleb (different TTS/VC systems, different speakers), DFDC (different video manipulation methods, 3,426 actors), FaceForensics++ (classic benchmark). - Quantify cross-dataset AUC and accuracy degradation. - Compare against published baselines on these benchmarks. - This directly measures the generalization that matters for deployment.

3. Full dataset training (most impactful for performance): - Use the complete AV-Deepfake1M++ training split (1M+ clips, 2,000+ speakers). - Requires: full dataset download (~ 1.4 TB), HDD storage, institutional GPU access. - Expected outcome: better generalization from greater speaker/generator diversity, possibly changing the $M3 > M2$ finding.

Q34: Temporal localization extension.

The AV-Deepfake1M++ dataset includes `fake_segments` — ground-truth temporal boundaries (`start_time`, `end_time`) of manipulation. Currently unused.

Sliding window approach (simplest): Slide a 2-second window across the full video with 50% overlap. Each window → model → joint score. Plot score as a time series. Windows overlapping `fake_segments` should score low; windows in real segments should score high. This requires no model changes — just inference-time pipeline modification.

Frame-level architecture (more accurate): Replace clip-level [CLS] prediction with per-token or per-frame predictions. Options: (a) Remove [CLS] token, use `video_feat` and `audio_feat` tokens directly → per-modality temporal predictions, (b) Extend Transformer to process video frame sequence tokens + audio time step tokens → cross-modal attention over time-aligned features. Requires architectural changes and retraining with temporal labels.

Evaluation metric: Temporal IoU between predicted and ground-truth `fake_segments`, standard in temporal action localization.

9. Ethics, Impact & Practicality

Q35: Ethical implications and misuse potential.

Positive impact: Detection contributes to harm mitigation — every improvement makes fraud, disinformation, and non-consensual content harder to execute. Per-modality interpretability assists forensic analysts. Open-source codebase supports reproducible research.

Misuse potential:

- **Adversarial training:** A generator can train against the detector (use detector’s loss as a GAN discriminator loss). My model, being trained on a specific dataset with known architecture, would be easier to adversarially target than a closed commercial system.
- **Over-reliance risk:** 93% accuracy with “zero false positives” (but wide CI: 86.3–96.6%) might encourage over-reliance in high-stakes contexts. The dissertation explicitly warns that FPR upper bound is ~14%.
- **Dataset consent:** AV-Deepfake1M++ involves real individuals whose likenesses were modified. Creators obtained consent; terms prohibit non-consensual use. I complied with all terms — detection research only, no redistribution.

Mitigation: Ethical approval (CN6000) confirmed secondary analysis of public benchmark requires no additional clearance. Appendix D documents dataset consent framework and compliance.

Q36: Is the system practically deployable?

Deployable aspects: Standalone inference (`inference.py`, no training deps), web interface with history/compare, resumable pipeline, interpretable output.

Gaps preventing deployment:

Gap	Severity	Fix
Unknown cross-dataset generalization	Critical	Evaluate on FakeAVCeleb, DFDC
No confidence intervals	High	Wilson CIs, MC Dropout uncertainty
100-video validation only	High	500 videos per type test set
No adversarial robustness testing	High	PGD/FGSM attacks; adversarial training
Fixed 2-second window	Medium	Sliding window with temporal scoring
No threshold optimization	Medium	Cost-sensitive calibration set
No deployment infrastructure	Medium	Docker, API gateway, scaling

MVP deployment path: Retrain on full dataset → cross-dataset validate → optimize threshold → add uncertainty → containerize.

Q37: Comparison to commercial tools.

Likely commercial advantages: Larger/diverse training data, continuously updated models, additional signals (metadata, provenance), team-scale engineering.

This system's advantages: Interpretable per-modality output (commercial tools typically give single score), transparent methodology (every design decision documented), academic rigor (speaker-disjoint evaluation, calibration analysis, explicit limitations).

Honest assessment: Research prototype demonstrating architectural ideas — cross-modal Transformer fusion, three-head interpretability, calibration analysis methodology. Not competitive with well-funded commercial detectors on raw performance, nor intended to be.

Q38: Broader significance of score calibration.

Score calibration is underappreciated in deepfake detection literature. Most papers report AUC and accuracy at ≈ 0.5 but rarely show reliability diagrams. My work demonstrates concretely: calibration can be the difference between deployable and non-deployable (93% vs 66% accuracy at identical AUC levels).

Recommendation for the field: Report ECE or reliability diagrams alongside AUC. Track calibration during training. This is a low-cost addition (one extra metric, one extra plot) providing substantially more deployment-relevant information than AUC alone. AUC measures ranking — calibration measures whether scores are meaningful as probabilities for threshold-based decisions.

Q39: Meta-lessons beyond technical results.

1. Engineering dominates architecture for applied ML. The most impactful decisions were engineering: torchaudio over librosa (handled corrupted MP4s), resumable manifests (survived cloud termination), two-phase training (protected pretrained features). Published papers focus on architecture; in practice, pipeline robustness often determines success.

2. Resource constraints shape conclusions. The 5-epoch cap, validation-split-only, 100-video test set were constraints, not choices. Conclusions are bounded by these constraints. Recognizing and stating limits is more honest than overclaiming.

3. Infrastructure management is a research skill. Managing cloud instances, SCP transfers, W&B logging, multiprocessing was more time-consuming than model design. Model 1's corrupted download — a 30-second check that wasn't done — was a lesson in operational diligence.

Quick Reference: Key Citations

Topic	Reference	Key Finding
AV-Deepfake1M++	Cai et al. (2025), <i>ACM MM '25</i>	2M clips, 4 types, 2K+ speakers
ResNet3D-18	Tran et al. (2018), arXiv:1711.11248	3D conv for spatiotemporal features
ResNet	He et al. (2015), arXiv:1512.03385	Residual learning; ImageNet pretraining

Topic	Reference	Key Finding
Focal Loss	Lin et al. (2018), arXiv:1708.02002	Down-weights easy examples via $(1-p_t)^\alpha$
Identity leakage	Rossler et al. (2019), <i>IEEE ICCV</i>	Random splits inflate accuracy 5–15pp
DFDC generalization failure	Dolhansky et al. (2020), arXiv:2006.07397	Cross-dataset AUC degradation 0.10–0.30
Audio deepfake survey	Yi et al. (2023), arXiv:2308.14970	Audio detection lags visual; multimodal needed
BERT / [CLS] token	Devlin et al. (2019), arXiv:1810.04805	Classification token aggregates sequence info
Neural net calibration	Guo et al. (2017), <i>ICML</i>	Modern networks less calibrated; temp scaling
Societal impact	Chesney & Citron (2019), <i>California Law Rev.</i>	Deepfakes: threat to privacy, democracy, security
SpecAugment	Park et al. (2019), <i>Interspeech</i>	Freq/time masking for spectrogram augmentation
Wav2Vec 2.0	Baevski et al. (2020), arXiv:2006.11477	Self-supervised speech representations
Kinetics-400	Kay et al. (2017), arXiv:1705.06950	400 human action categories; 306K videos
AD-NeRF	Guo et al. (2021), <i>IEEE ICCV</i>	Neural talking-head synthesis
NaturalSpeech 2	Shen et al. (2023), arXiv:2304.09116	Zero-shot voice cloning via diffusion
Kinetics pretrained	Kay et al. (2017)	Talking, singing, gesturing categories relevant
Codebase	github.com/Jasmipreethi/Deepfake	Full pipeline: 790-line main.py + 13 modules